```
# Palatial Pools | Calendar System
**Proprietary Technology**
**Confidential**

---

## Table of Contents

1. [Introduction](#introduction)
2. [System Architecture](#system-architecture)
3. [Core Functional Modules](#core-functional-modules)
   - [Scheduling Engine](#scheduling-engine)
   - [Route Optimisation](#route-optimisation)
   - [Customer Communication Hub Integration](#customer-communication-hub-integration)
4. [Data Models and Formats](#data-models-and-formats)
5. [Security Architecture](#security-architecture)
6. [Infrastructure and Deployment](#infrastructure-and-deployment)
7. [Integration and Extensibility](#integration-and-extensibility)
8. [Compliance and Standards](#compliance-and-standards)
9. [Appendices](#appendices)
   - [A. Sample API Specification](#appendix-a-sample-api-specification)
   - [B. Algorithmic Overview](#appendix-b-algorithmic-overview)


---

## Introduction

The **Palatial Pools Calendar System** is a comprehensive scheduling and workforce management platform engineered specifically for pool service operations. Designed to integrate customer management, job specifications, technician availability, and route optimisation into a single cohesive platform, the system aims to streamline field operations, reduce operational costs, and improve customer service quality.

This document provides an in-depth technical description of the Calendar System, covering its architecture, core algorithms, data handling, security measures, and integration capabilities. It is intended for engineers, IT architects, prospective investors, and technical decision makers evaluating the platform for deployment or partnership.

---

## System Architecture
```

```
The Calendar System is developed as a cloud-native, distributed application composed of microservices and client interfaces optimized for desktop and mobile environments. The system is designed for high availability, low latency, and secure data handling.

### High-Level Architecture Diagram

```mermaid
graph TD
    A[Customer Records DB] -->|Job Requests| B[Scheduling Engine]
    B --> C[Route Optimisation Engine]
    B --> D[Notification Service]
    D --> E[Comms Hub Integration]
    C --> F[Technician Mobile App]
    B --> F
    G[Admin Interface] --> B
    H[Audit Log DB] --> B
    subgraph Cloud Backend
      B
      C
      D
      E
      H
    end
    subgraph Client Devices
      F
      G
    end
```

Architectural Components

- **Customer Records Database**: Stores detailed customer profiles, service history, preferences, and job specifications.
- **Scheduling Engine**: Core module responsible for appointment creation, modification, and technician assignment.
- **Route Optimisation Engine**: Computes optimal daily routes for technicians using geographical data.
- **Notification Service**: Generates and dispatches customer notifications including appointment confirmations and reminders.
```

- **Comms Hub**: Integration layer managing multi-channel communications (SMS, email, push notifications).
- **Technician Mobile Application**: Frontend interface for technicians to receive schedules, update job statuses, and communicate.
- **Admin Interface**: Web-based dashboard for scheduling staff and management.
- **Audit Log Database**: Immutable log capturing all schedule changes and system events for compliance and troubleshooting.

Core Functional Modules

Scheduling Engine

Overview

The Scheduling Engine automates job assignment by correlating multiple constraints and preferences:

- **Technician Skills Matrix**: Each technician profile includes certifications, skill levels, and equipment proficiencies.
- **Equipment Availability**: Jobs requiring specialised equipment ensure assignment only to technicians equipped accordingly.
- **Geospatial Considerations**: Technician proximity to job site is factored into scheduling decisions.
- **Customer Preferences**: Preferred time windows and other constraints are respected where possible.
- **Business Rules**: Configurable rules such as maximum daily workload, break times, and job priorities.

Scheduling Algorithm

The engine implements a constraint satisfaction problem (CSP) solver enhanced with heuristics for practical tractability:

- **Input**: Job list, technician availability, constraints.

- **Method**: Backtracking search with forward checking and constraint propagation.
- **Heuristics**:
  - Minimum Remaining Values (MRV) for job assignment order.
  - Least Constraining Value (LCV) to select technician assignments minimizing future conflicts.
- **Output**: Feasible schedule with assigned technicians and time slots.

Drag-and-Drop Rescheduling

User-initiated rescheduling via drag-and-drop on the calendar interface triggers:

- Conflict detection using interval tree data structure.
- Automatic recalculation of downstream dependencies.
- Immediate update propagation via WebSocket connections to all relevant clients and the technician app.
- Triggering customer notification workflows if appointment parameters change.

Route Optimisation

Purpose

Minimises travel time and fuel consumption by optimally sequencing jobs assigned to a technician.

Algorithmic Approach

The system employs a hybrid approach combining:

- **Initial solution generation** via Nearest Neighbor heuristic to rapidly produce a feasible route.
- **Route improvement** using a metaheuristic such as 2-opt or Lin-Kernighan heuristic to reduce total distance/time.
- **Time window constraints** incorporated using a Time-Dependent Vehicle Routing Problem (TDVRP) model.

The optimisation leverages geographic coordinates (latitude/longitude) retrieved from the customer address database, processed using the Haversine formula for accurate

distance estimation.

External Data Sources

- Integration with third-party map APIs (e.g., Google Maps API, HERE Maps) for real-time traffic data and routing.
- Periodic updates to road network graphs ensure route accuracy.

Customer Communication Hub Integration

The Calendar System integrates tightly with the **Comms Hub** to automate customer interaction workflows.

Communication Channels

- SMS (using standards such as SMPP or REST APIs)
- Email (SMTP with TLS 1.3 encryption)
- Push notifications via mobile apps (using Firebase Cloud Messaging or Apple Push Notification Service)

Notification Workflow

1. Appointment creation triggers an event published on an internal message bus (Kafka/RabbitMQ).
2. Notification Service consumes event, formats messages according to templates.
3. Messages are dispatched asynchronously with retry and failure handling.
4. Delivery confirmation is logged for audit and compliance.

Data Models and Formats

Customer Record

```
{
  "customer_id": "UUID-v4",
  "name": "John Doe",
  "address": {
    "street": "123 Poolside Ave",
    "suburb": "Sydney",
    "postcode": "2000",
    "state": "NSW",
    "coordinates": {
      "lat": -33.8688,
      "lng": 151.2093
    }
  },
  "contact": {
    "email": "john.doe@example.com",
    "phone": "+61 2 9876 5432"
  },
  "preferences": {
    "appointment_windows": [
      {"start": "09:00", "end": "12:00"},
      {"start": "14:00", "end": "17:00"}
    ]
  },
  "service_history": [
    {
      "job_id": "UUID-v4",
      "service_date": "2024-05-01T09:00:00Z",
      "service_type": "Weekly Maintenance",
      "technician_id": "UUID-v4"
    }
  ]
}
```

Job Specification

- Job ID (UUID)
- Customer ID (UUID)

- Service Type (Enum: e.g., Weekly Maintenance, Equipment Repair, Chemical Balancing)
- Estimated Duration (minutes)
- Equipment Required (List of equipment IDs)
- Priority Level (Integer scale)
- Status (Scheduled, In Progress, Completed, Cancelled)
- Scheduled Time Window (Start/End ISO 8601 datetime)

Technician Profile

- Technician ID (UUID)
- Name
- Skills (Set of skill identifiers)
- Certifications (List of certification documents, expiry dates)
- Equipment Assigned
- Availability Calendar (Free/busy time slots)

Data Formats

- All data exchanges use JSON over HTTPS REST APIs.
- Real-time synchronization uses WebSocket connections with messages encoded in JSON.
- Date/time values conform to ISO 8601 with timezone offsets.
- Geocoordinates use WGS84 datum (EPSG:4326).

Security Architecture

Data Protection

- **Encryption at Rest**: Customer and schedule data encrypted using AES-256 GCM mode on database storage.

- **Encryption in Transit**: All communications between client devices and backend use TLS 1.3, ensuring forward secrecy and integrity.
- **Role-Based Access Control (RBAC)**: Access to scheduling data is controlled at the granularity of roles (e.g., Admin, Scheduler, Technician), enforced via OAuth 2.0 Bearer tokens.
- **Audit Trails**: Immutable logs capture all changes with timestamps, user IDs, and before/after states. Logs are digitally signed and stored with write-once-read-many (WORM) policies.

Authentication and Authorization

- Authentication performed via secure OAuth 2.0/OpenID Connect flows.
- Multi-factor authentication (MFA) enabled for administrative users.
- Session management includes idle timeouts and token revocation mechanisms.

Compliance Considerations

- Adheres to Australian Privacy Principles (APPs) under the Privacy Act 1988.
- Data retention policies comply with relevant state and federal legislation.
- Customer notification opt-in/out settings are managed per Australian Spam Act 2003.

Infrastructure and Deployment

Deployment Model

- Cloud-hosted on Australian data centers compliant with the Australian Signals Directorate (ASD) Essential Eight cybersecurity strategies.
- Scalable microservices deployed in containerized environments (Docker, Kubernetes).
- Load balancing implemented with ingress controllers and API gateways.
- High availability ensured via multi-zone deployment with automatic failover.

Real-Time Synchronization

- WebSocket-based event bus connects clients and backend for immediate updates.
- Conflict resolution is handled via optimistic concurrency controls and versioning.
- Data synchronization latency target: <200ms under typical load.

Integration with Pooltrackr

- Bi-directional synchronization of pool equipment and maintenance records.
- RESTful API endpoints with token-based authentication.
- Webhook subscriptions notify Calendar System of equipment status changes.

Network and Protocols

- Backend services communicate internally over gRPC with mTLS (mutual TLS) authentication.
- External client connections utilize HTTPS/REST and WebSocket over TLS 1.3.
- Internal databases use encrypted connections (e.g., PostgreSQL with SSL).

Integration and Extensibility

API Overview

- Comprehensive REST API exposing endpoints for customer management, scheduling, technician management, notifications, and reporting.
- API follows RESTful principles with resource-oriented URLs.
- Supports pagination, filtering, and sorting on list endpoints.
- Rate limiting applied to prevent abuse.

Webhooks

- Supports outbound webhooks for event notifications (e.g., job status changes).

- Webhook payloads use JSON schema with signature verification via HMAC SHA-256.

SDKs and Developer Tools

- SDKs provided in JavaScript and Python for integration ease.
- API documentation published using OpenAPI 3.0 specification.
- Sandbox environment available for development and testing.

Compliance and Standards

- **Data Security**: TLS 1.3 (RFC 8446), AES-256 encryption standards.
- **Authentication**: OAuth 2.0 (RFC 6749), OpenID Connect.
- **Data Formats**: JSON (RFC 8259), ISO 8601 date/time.
- **Geospatial**: WGS84 coordinate system (EPSG:4326).
- **Communication**: SMPP for SMS, SMTP with STARTTLS for email.
- **Privacy**: Compliant with Australian Privacy Principles (APPs).
- **Accessibility**: Interfaces designed to meet WCAG 2.1 AA standards.

Appendices

Appendix A: Sample API Specification (Excerpt)

```
openapi: 3.0.0
info:
  title: Palatial Pools Calendar System API
  version: '1.0'
paths:
  /customers/{customerId}/jobs:
    get:
      summary: Retrieve list of jobs for a customer
      parameters:
        - name: customerId
          in: path
          required: true
          schema:
            type: string
            format: uuid
      responses:
        '200':
          description: List of jobs
          content:
            application/json:
              schema:
                type: array
                items:
                  $ref: '#/components/schemas/Job'
components:
  schemas:
    Job:
      type: object
      properties:
        job_id:
          type: string
          format: uuid
        service_type:
          type: string
        scheduled_time:
          type: string
          format: date-time
        status:
```

```
          type: string
          enum: [Scheduled, In Progress, Completed, Cancelled]
```

Appendix B: Algorithmic Overview

Constraint Satisfaction Problem (CSP) Formulation for Scheduling

- **Variables**: Each job to be scheduled.
- **Domains**: Possible technician/time slot combinations.
- **Constraints**:
  - Technician skill compatibility.
  - Availability time windows.
  - Equipment requirements.
  - Customer preferences.
  - Non-overlapping schedules.

The CSP solver applies:

- **Backtracking Search**: Explore assignments, backtracking on conflicts.
- **Forward Checking**: Prune inconsistent values early.
- **Heuristics**: MRV and LCV to improve efficiency.

Route Optimisation Algorithm Pseudocode

```
Input: List of jobs with locations and time windows
Output: Ordered route minimizing travel time and respecting constraints

1. Initialize route with starting location (technician base)
2. While unvisited jobs remain:
    a. Select nearest feasible job respecting time windows
    b. Append job to route
3. Apply 2-opt optimization:
    a. For each pair of edges, try swapping to reduce total distance
    b. Repeat until no improvement
4. Return optimized route
```

End of Document